```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
url = (
    "https://archive.ics.uci.edu/ml/"
    "machine-learning-databases/00275/"
    "Bike-Sharing-Dataset.zip"
)

# Download and extract the dataset
import zipfile
import io
import requests

response = requests.get(url)
response.raise_for_status()

with zipfile.ZipFile(io.BytesIO(response.content)) as z:
    with z.open("day.csv") as file:
        data = pd.read_csv(file)

print("Dataset loaded successfully!")
print("Dataset shape:", data.shape)

display(data.head())
```

```
Dataset loaded successfully!
Dataset shape: (731, 16)
```

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	
0	1	2011-01-01	1	0	1	0	6	0	2	0
1	2	2011-01-02	1	0	1	0	0	0	2	0
2	3	2011-01-03	1	0	1	0	1	1	1	0

```
print("Dataset Information:")
data.info()
```

```
print("\nMissing Values:")
print(data.isnull().sum())

print("\nStatistical Summary:")
display(data.describe())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 731 entries, 0 to 730
Data columns (total 16 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   instant     731 non-null    int64
```

```
# Convert date column into datetime format
data["dteday"] = pd.to_datetime(data["dteday"])

# Create readable season names
season_map = {
    1: "Spring",
    2: "Summer",
    3: "Fall",
    4: "Winter"
}

data["season_name"] = data["season"].map(season_map)

# Create readable weekday names
data["weekday_name"] = data["weekday"].map({
    0: "Sunday",
    1: "Monday",
    2: "Tuesday",
    3: "Wednesday",
    4: "Thursday",
    5: "Friday",
    6: "Saturday"
})

print("Data cleaning completed!")
display(data.head())
```

```
atemp          0
hum            0
windspeed      0
casual         0
dtype: int64
Statistical Summary:
```

Data cleaning completed!

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	
0	1	2011-01-01	1	0	1	0	6	0	2	0
1	2	2011-01-02	1	0	1	0	0	0	2	0
2	3	2011-01-03	1	0	1	0	1	1	1	0
3	4	2011-01-04	1	0	1	0	2	1	1	0
4	5	2011-01-05	1	0	1	0	3	1	1	0

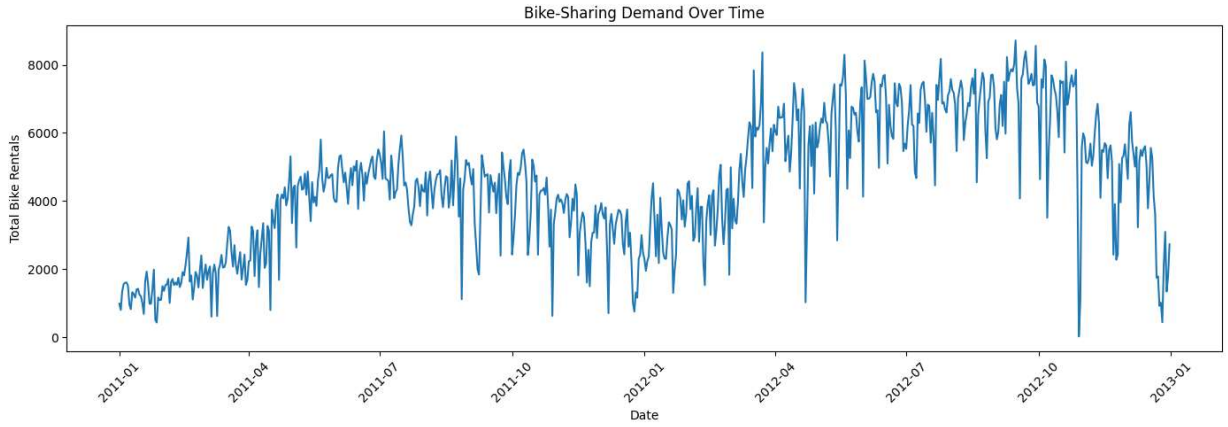
mean	366.000000	2.496580	0.500684	6.519836	0.028728	2.997264
25%	183.500000	2.000000	0.000000	4.000000	0.000000	1.000000

```
plt.figure(figsize=(14, 5))

plt.plot(data["dteday"], data["cnt"])

plt.title("Bike-Sharing Demand Over Time")
plt.xlabel("Date")
plt.ylabel("Total Bike Rentals")
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```



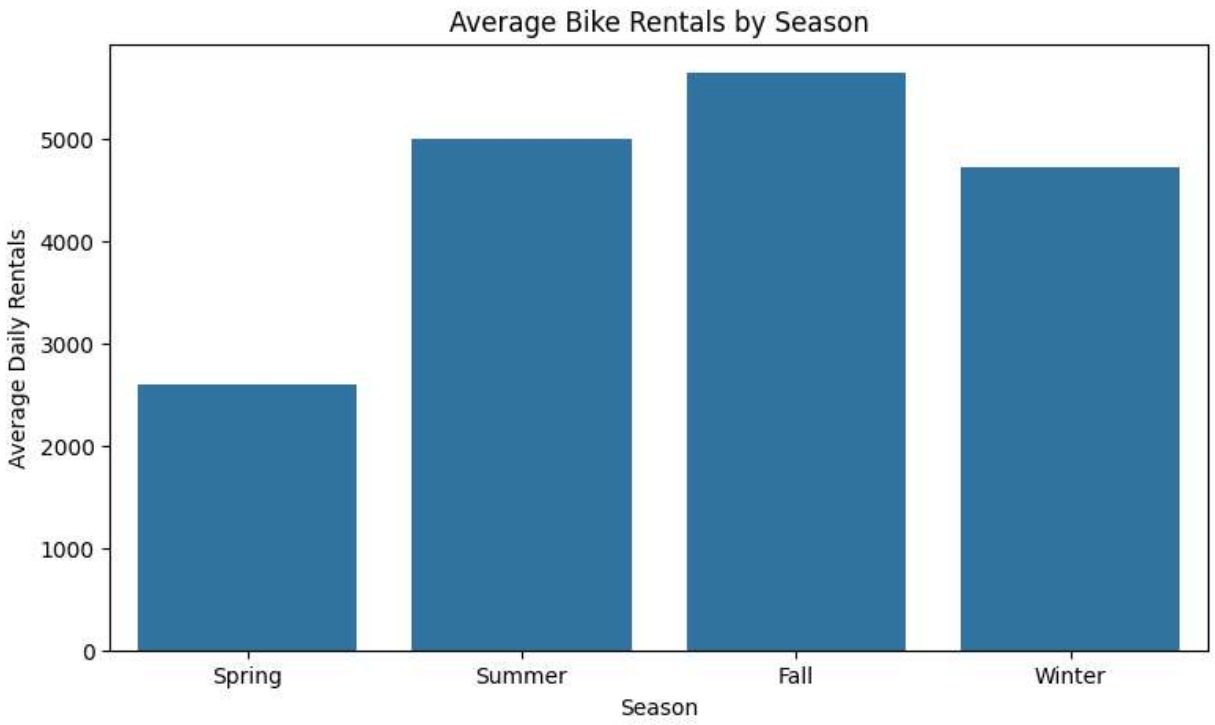

```
plt.figure(figsize=(9, 5))

sns.barplot(
    data=data,
    x="season_name",
    y="cnt",
    estimator="mean",
    errorbar=None
)

plt.title("Average Bike Rentals by Season")
plt.xlabel("Season")
```

```
plt.ylabel("Average Daily Rentals")

plt.show()
```

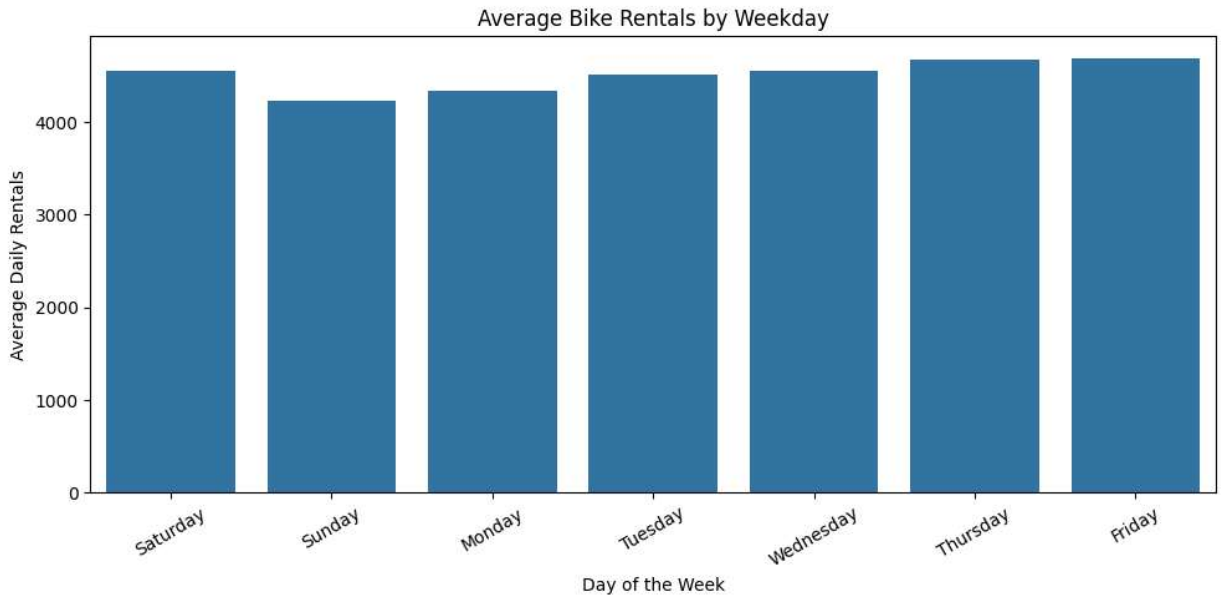



```
plt.figure(figsize=(10, 5))

sns.barplot(
    data=data,
    x="weekday_name",
    y="cnt",
    estimator="mean",
    errorbar=None
)

plt.title("Average Bike Rentals by Weekday")
plt.xlabel("Day of the Week")
plt.ylabel("Average Daily Rentals")
plt.xticks(rotation=30)

plt.tight_layout()
plt.show()
```

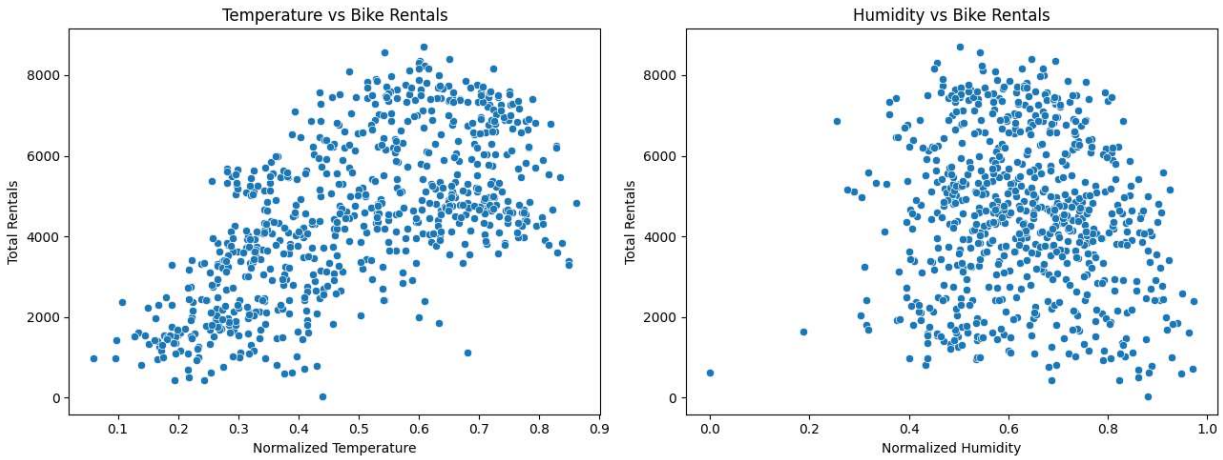



```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

sns.scatterplot(
    data=data,
    x="temp",
    y="cnt",
    ax=axes[0]
)
axes[0].set_title("Temperature vs Bike Rentals")
axes[0].set_xlabel("Normalized Temperature")
axes[0].set_ylabel("Total Rentals")

sns.scatterplot(
    data=data,
    x="hum",
    y="cnt",
    ax=axes[1]
)
axes[1].set_title("Humidity vs Bike Rentals")
axes[1].set_xlabel("Normalized Humidity")
axes[1].set_ylabel("Total Rentals")
```

```
plt.tight_layout()
plt.show()
```

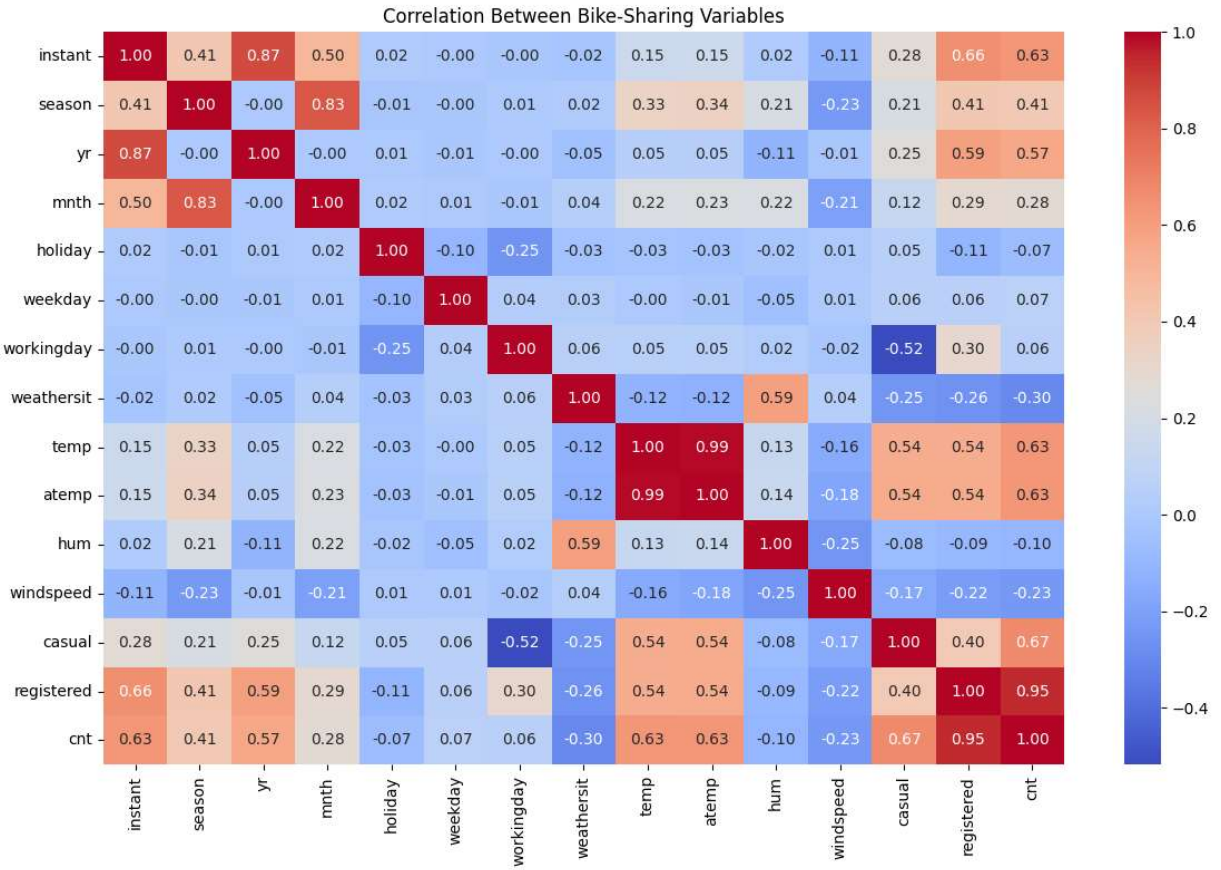



```
numeric_data = data.select_dtypes(include="number")

plt.figure(figsize=(12, 8))

sns.heatmap(
    numeric_data.corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Between Bike-Sharing Variables")
plt.tight_layout()
plt.show()
```




```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
```

```
# Select input features
features = [
    "season", "yr", "mnth", "holiday",
    "weekday", "workingday", "weathersit",
    "temp", "atemp", "hum", "windspeed"
]

X = data[features]
y = data["cnt"]

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Train the model
model = RandomForestRegressor(
    n_estimators=200,
    random_state=42
)

model.fit(X_train, y_train)

# Predict demand
y_pred = model.predict(X_test)

print("Model trained successfully!")
print("MAE:", round(mean_absolute_error(y_test, y_pred), 2))
print("RMSE:", round(np.sqrt(mean_squared_error(y_test, y_pred)), 2))
print("R² Score:", round(r2_score(y_test, y_pred), 3))
```

```
Model trained successfully!
MAE: 426.41
RMSE: 672.21
R² Score: 0.887
```

```
importance = pd.Series(
    model.feature_importances_,
    index=features
).sort_values(ascending=False)

print("Factors influencing bike-sharing demand:")
print(importance)

plt.figure(figsize=(10, 6))

importance.sort_values().plot(kind="barh")
```

```
plt.title("Feature Importance for Bike-Sharing Demand")
plt.xlabel("Importance")
plt.ylabel("Feature")

plt.tight_layout()
plt.show()
```

```
Factors influencing bike-sharing demand:
temp          0.350069
yr            0.280666
atemp         0.151281
hum           0.058661
season        0.055358
windspeed     0.031977
mnth          0.029561
weathersit    0.019084
weekday       0.015958
workingday    0.004620
holiday       0.002766
dtype: float64
```

